REPORT OF ASSIGNMENT 1

Name: Rakshit Rathour
Roll No: 2501630020
Program: BTech CSE (Robotics & AI)
Subject: Data Structures

Part A: Stack ADT

What is an ADT?

An **Abstract Data Type (ADT)** defines a data structure based on the **operations that can be performed on it**, without describing how those operations are internally implemented.

For example, a **Stack ADT** supports operations such as:

- **Push** – add an element to the top of the stack
- **Pop** – remove the top element
- **Peek / Top** – view the top element without removing it
- **isEmpty** – check whether the stack has any elements

The key concept is that users interact only with these operations. Internally, the stack can be implemented using structures like **arrays or linked lists**, but the behavior from the user's perspective remains the same.

Efficiency of Stack Operations

Stack operations such as **push** and **pop** are efficient because they only involve the **top element** of the stack. Since no other elements need to be moved or rearranged, these operations run in **constant time O(1)**.

Part B & Part D: Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Paradigm
Factorial	O(n)	O(n)	Recursion
Fibonacci (Naive)	$O(2^n)$	O(n)	Recursion
Fibonacci (Memoized)	O(n)	O(n)	Dynamic Programming
Tower of Hanoi	$O(2^n)$	O(n)	Recursion

Algorithm	Time Complexity	Space Complexity	Paradigm
Binary Search	O(log n)	O(log n)	Divide and Conquer

Note on the Fibonacci Algorithm

The basic recursive Fibonacci approach is inefficient because it **recalculates the same values multiple times**.

For example, when calculating **fib(5)**, the algorithm computes **fib(4)** and **fib(3)**. However, **fib(4)** again requires **fib(3)**, resulting in repeated work. As the input value increases, the number of repeated calculations grows rapidly, giving the algorithm an **exponential time complexity of $O(2^n)$**.

Binary Search Recurrence Relation

Binary Search reduces the search space by **half at every step**.

The recurrence relation is:

$T(n) = T(n/2) + O(1)$

This indicates that at each step the problem size becomes half, while only a constant amount of work is required for comparison.

Solving this relation results in the overall complexity:

$T(n) = O(\log n)$

Part E: Paradigm Reflection

1. Binary Search – Divide and Conquer

Binary Search applies the **Divide and Conquer** technique by repeatedly splitting the problem into smaller parts.

Divide

The algorithm first determines the **middle element** of the sorted array and compares it with the target value.

Conquer

Based on the result of the comparison:

- If the target value is **smaller than the middle element**, the search continues in the **left half** of the array.
- If the target value is **greater**, the search proceeds in the **right half**.

- If the middle element matches the target value, the search ends.

This strategy greatly reduces the number of elements that need to be checked.

Case Analysis

Best Case
O(1) – the required element is found at the middle position immediately.

Average Case
O(log n)

Worst Case
O(log n) – the array keeps getting divided until only one element remains.

Example

A real-world example of Binary Search is **looking up a word in a dictionary**. Instead of checking every page, you open the book somewhere near the middle and decide whether the word lies before or after that point. You then repeat the process on the appropriate half until the word is located.

---

2. Fibonacci with Memoization (Dynamic Programming)

The Problem: Overlapping Subproblems

In the simple recursive Fibonacci method, many values are calculated multiple times.

Example:

- fib(5) requires fib(4) and fib(3)
- fib(4) again requires fib(3) and fib(2)

This repetition causes the number of function calls to grow very quickly.

Memoization Technique

Memoization improves performance by **storing results of previously computed values**.

Steps involved:

1. Before computing fib(n), check if the result already exists.
2. If it exists, return the stored value.
3. Otherwise compute the value and store it for later use.

This ensures that each Fibonacci value is calculated only once.

Connection to Dynamic Programming

Memoization is a **Dynamic Programming** technique where a complex problem is broken into smaller subproblems and each subproblem is solved only once. This approach is often referred to as **Top-Down Dynamic Programming**.

Efficiency Improvement

Method	Time Complexity
Naive Fibonacci	$O(2^n)$
Memoized Fibonacci	$O(n)$

By saving intermediate results, the algorithm becomes much faster.

Example

A similar concept is used in **navigation systems like GPS**. When calculating routes between locations, the system stores results of smaller path segments. If the same segment is needed again, it can reuse the stored result instead of recalculating it.

---

3. Tower of Hanoi and Recursion

The **Tower of Hanoi** problem demonstrates how recursion solves problems by dividing them into smaller tasks.

Smaller Subproblem Concept

To move **n disks** from the source peg to the destination peg:

1. Move the top **n−1 disks** from the source peg to the auxiliary peg.
2. Move the **largest disk** to the destination peg.
3. Move the **n−1 disks** from the auxiliary peg to the destination peg.

Thus, the main problem is reduced to smaller instances of the same problem.

Base Case

The recursion stops when **n = 1**.
In this case, the single disk can be moved directly from the source peg to the destination peg.

Complexity of Tower of Hanoi

Total Number of Moves

$2^n - 1$

Time Complexity

$O(2^n)$

Space Complexity

$O(n)$

The space complexity represents the **maximum depth of recursive calls stored in the call stack**.